1. Apply PCA on the Wine dataset to reduce dimensionality and visualize principal components for distinguishing between red and white wine.

In [9]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

In [6]:
```
df=pd.read_csv("winequality.csv")
df.sample(10)
```

Out[6]:

	type	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	d
2657	white	6.45	0.140	0.42	1.2	0.050	51.0	129.0	0
5591	red	9.00	0.480	0.32	2.8	0.084	21.0	122.0	0.
598	white	5.90	0.270	0.29	11.4	0.036	31.0	115.0	0.
95	white	7.10	0.260	0.29	12.4	0.044	62.0	240.0	0.
2879	white	5.70	0.370	0.30	1.1	0.029	24.0	88.0	0.
595	white	6.30	0.270	0.38	0.9	0.051	7.0	140.0	0.
4943	red	4.60	0.520	0.15	2.1	0.054	8.0	65.0	0.
5941	red	9.50	0.390	0.41	8.9	0.069	18.0	39.0	0.
6179	red	7.10	0.460	0.20	1.9	0.077	28.0	54.0	0.
5028	red	8.00	0.745	0.56	2.0	0.118	30.0	134.0	0.

In [7]:
```
df.shape
```

Out[7]:
```
(6497, 13)
```

In [8]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6497 entries, 0 to 6496
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   type                  6497 non-null   object 
 1   fixed acidity         6487 non-null   float64
 2   volatile acidity      6489 non-null   float64
 3   citric acid           6494 non-null   float64
 4   residual sugar        6495 non-null   float64
 5   chlorides             6495 non-null   float64
 6   free sulfur dioxide   6497 non-null   float64
 7   total sulfur dioxide  6497 non-null   float64
 8   density               6497 non-null   float64
 9   pH                    6488 non-null   float64
 10  sulphates             6493 non-null   float64
 11  alcohol               6497 non-null   float64
 12  quality               6497 non-null   int64  
dtypes: float64(11), int64(1), object(1)
memory usage: 660.0+ KB
```

In [11]:
```
df.isnull().sum()
```

Out[11]:
```
type                    0
fixed acidity          10
volatile acidity        8
citric acid             3
residual sugar          2
chlorides               2
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      9
sulphates               4
alcohol                 0
quality                 0
dtype: int64
```

In [23]:
```
X = df.drop(columns=["type", "quality"])
y = df["type"]

print("Features shape:", X.shape)
print("Labels shape:", y.shape)
```

```
Features shape: (6497, 11)
Labels shape: (6497,)
```

In [24]:
```
X = X.fillna(X.median())
```

In [25]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [29]:
```
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
```

```
print("PCA output shape:", X_pca.shape)
print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Total variance retained:",pca.explained_variance_ratio_.sum(
```

```
PCA output shape: (6497, 2)
Explained variance ratio: [0.27540156 0.22670082]
Total variance retained: 50.21023750880532 %
```

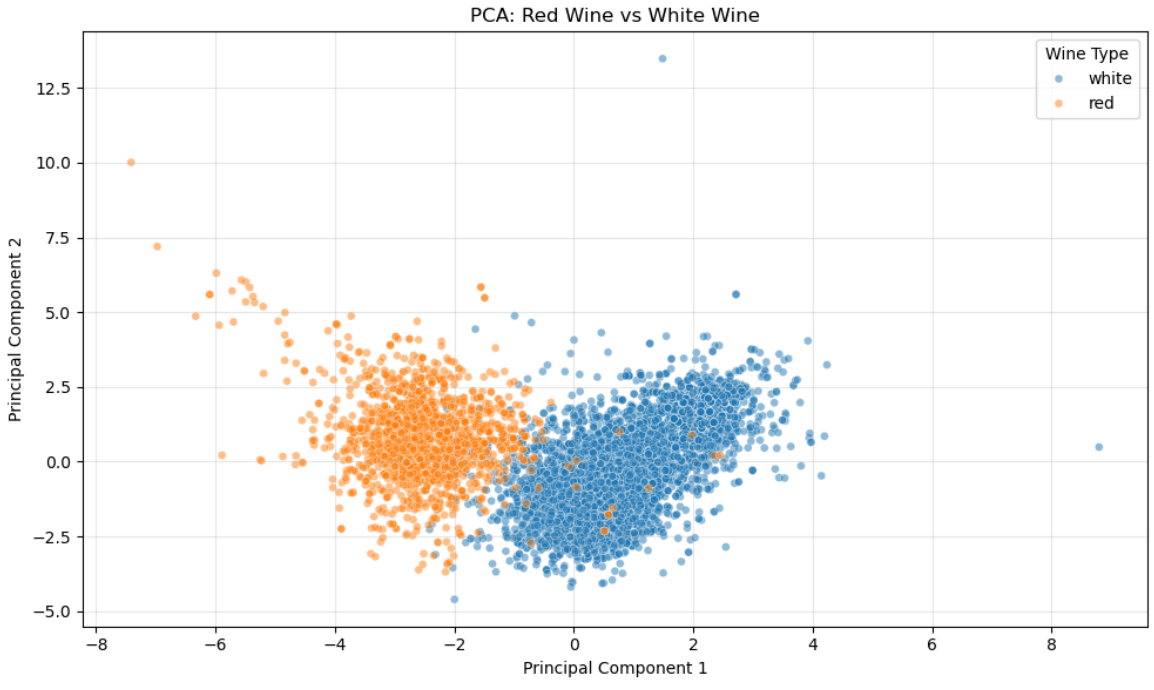
In [27]:
```
pca_df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)

pca_df["type"] = y.reset_index(drop=True)

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=pca_df,
    x="PC1",
    y="PC2",
    hue="type",
    alpha=0.5,
    s=25
)

plt.title("PCA: Red Wine vs White Wine")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Wine Type")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



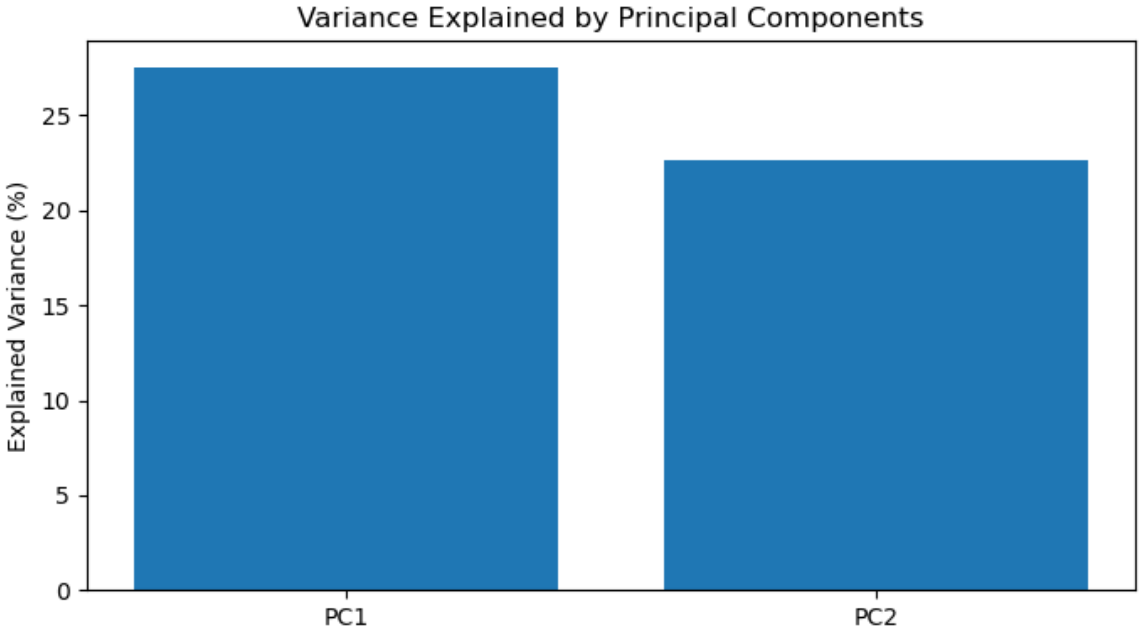
In [28]:
```
variance = pca.explained_variance_ratio_ * 100

plt.figure(figsize=(7, 4))
```

```
plt.bar(
    ["PC1", "PC2"],
    variance
)

plt.ylabel("Explained Variance (%)")
plt.title("Variance Explained by Principal Components")
plt.tight_layout()
plt.show()

print(f"PC1: {variance[0]:.2f}%")
print(f"PC2: {variance[1]:.2f}%")
print(f"Combined: {variance.sum():.2f}%")
```




```
PC1: 27.54%
PC2: 22.67%
Combined: 50.21%
```

2. Develop a Ridge and Lasso regression model to predict the number of bike rentals based on weather conditions and time.

In [35]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error, r2_score
```

In [32]:
```
df=pd.read_csv("bike.csv")
```

In [33]:
```
df.sample(10)
```

Out[33]:

	instant	dteday	season	yr	mnth	hr	holiday	weekday	workingda
2579	2580	2011-04-22	2	0	4	9	0	5	
9235	9236	2012-01-25	1	1	1	17	0	3	
4203	4204	2011-06-29	3	0	6	1	0	3	
10771	10772	2012-03-29	2	1	3	22	0	4	
17277	17278	2012-12-27	1	1	12	18	0	4	
12895	12896	2012-06-26	3	1	6	12	0	2	
6243	6244	2011-09-22	3	0	9	17	0	4	
6811	6812	2011-10-16	4	0	10	9	0	0	
8140	8141	2011-12-10	4	0	12	20	0	6	
7532	7533	2011-11-15	4	0	11	11	0	2	

In [34]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17379 entries, 0 to 17378
Data columns (total 17 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   instant     17379 non-null  int64  
 1   dteday      17379 non-null  object 
 2   season      17379 non-null  int64  
 3   yr          17379 non-null  int64  
 4   mnth        17379 non-null  int64  
 5   hr          17379 non-null  int64  
 6   holiday     17379 non-null  int64  
 7   weekday     17379 non-null  int64  
 8   workingday  17379 non-null  int64  
 9   weathersit  17379 non-null  int64  
 10  temp        17379 non-null  float64
 11  atemp       17379 non-null  float64
 12  hum         17379 non-null  float64
 13  windspeed   17379 non-null  float64
 14  casual      17379 non-null  int64  
 15  registered  17379 non-null  int64  
 16  cnt         17379 non-null  int64  
dtypes: float64(4), int64(12), object(1)
memory usage: 2.3+ MB
```

In [36]:
```
df.isnull().sum()
```

Out[36]:
```
instant       0
dteday        0
season        0
yr            0
mnth          0
hr            0
holiday       0
weekday       0
workingday    0
weathersit    0
temp          0
atemp         0
hum           0
windspeed     0
casual        0
registered    0
cnt           0
dtype: int64
```

In [37]:
```
features = ["season","yr","mnth","hr","holiday","weekday",
               "workingday","weathersit","temp","atemp","hum","wind
```

In [38]:
```
X = df[features]
y = df["cnt"]
```

In [39]:
```
print("Input shape:", X.shape)
print("Target shape:", y.shape)
```

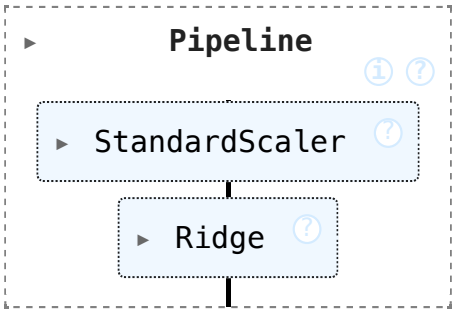
```
Input shape: (17379, 12)
Target shape: (17379,)
```

In [53]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,shuffle=False
)
```

In [54]:
```
ridge_model = make_pipeline(
    StandardScaler(),
    Ridge(alpha=1.0)
)
```

In [55]:
```
ridge_model.fit(X_train, y_train)
```

Out[55]:

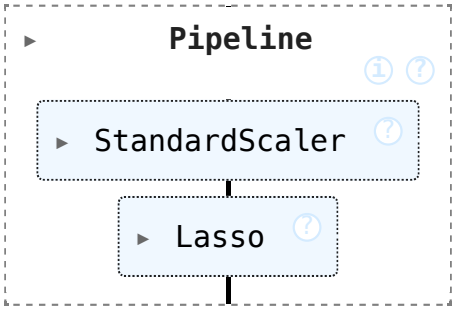


In [56]:
```
ridge_pred = ridge_model.predict(X_test)
```

In [57]:
```
lasso_model = make_pipeline(
    StandardScaler(),
    Lasso(alpha=1.0, max_iter=10000)
)
```

In [58]:
```
lasso_model.fit(X_train, y_train)
```

Out[58]:



In [59]:
```
lasso_pred = lasso_model.predict(X_test)
```

In [60]:
```
def evaluate_model(name, actual, predicted):
    mae = mean_absolute_error(actual, predicted)
    rmse = np.sqrt(mean_squared_error(actual, predicted))
    r2 = r2_score(actual, predicted)

    print(name)
    print("MAE:", round(mae, 2))
    print("RMSE:", round(rmse, 2))
    print("R2 Score:", round(r2, 4))
    print()
```

In [61]:
```
evaluate_model("Ridge Regression", y_test, ridge_pred)
```

```
Ridge Regression
MAE: 138.29
RMSE: 183.28
R2 Score: 0.309
```

In [62]:
```
evaluate_model("Lasso Regression", y_test, lasso_pred)
```

```
Lasso Regression
MAE: 137.67
RMSE: 183.99
R2 Score: 0.3037
```

In [63]:
```
plt.figure(figsize=(12, 5))

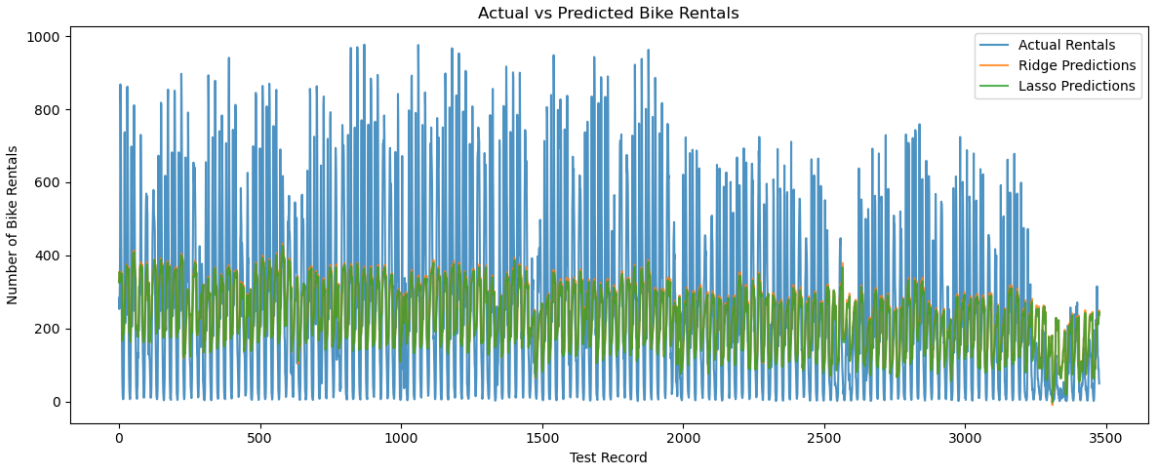
plt.plot(
    range(len(y_test)),
    y_test.values,
    label="Actual Rentals",
    alpha=0.8
)

plt.plot(
```

```
    range(len(y_test)),
    ridge_pred,
    label="Ridge Predictions",
    alpha=0.8
)

plt.plot(
    range(len(y_test)),
    lasso_pred,
    label="Lasso Predictions",
    alpha=0.8
)

plt.xlabel("Test Record")
plt.ylabel("Number of Bike Rentals")
plt.title("Actual vs Predicted Bike Rentals")
plt.legend()
plt.tight_layout()
plt.show()
```



In [64]:
```
results = pd.DataFrame({
    "Model": ["Ridge", "Lasso"],
    "MAE": [
        mean_absolute_error(y_test, ridge_pred),
        mean_absolute_error(y_test, lasso_pred)
    ],
    "RMSE": [
        np.sqrt(mean_squared_error(y_test, ridge_pred)),
        np.sqrt(mean_squared_error(y_test, lasso_pred))
    ],
    "R2 Score": [
        r2_score(y_test, ridge_pred),
        r2_score(y_test, lasso_pred)
    ]
})

print(results.round(4).to_string(index=False))
```

```
Model      MAE     RMSE  R2 Score
Ridge 138.2918 183.2793    0.3090
Lasso 137.6658 183.9861    0.3037
```

3. Create a multiclass classification model to predict wine quality based on chemical properties.

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import ConfusionMatrixDisplay
```

In [2]:
```
df=pd.read_csv("winequality.csv")
```

In [3]:
```
df.sample(5)
```

Out[3]:

	type	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	d
5840	red	10.1	0.43	0.40	2.6	0.092	13.0	52.0	0.
4991	red	7.7	0.49	0.26	1.9	0.062	9.0	31.0	0.
3211	white	8.2	0.30	0.44	12.4	0.043	52.0	154.0	0.
4018	white	6.4	0.18	0.48	18.0	0.054	56.0	183.0	1.
1247	white	7.2	0.21	0.34	1.1	0.046	25.0	80.0	0.

In [4]:
```
df.shape
```

Out[4]:
```
(6497, 13)
```

In [5]:
```
X = df.drop(columns=["quality", "type"])
y = df["quality"]
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

In [7]:
```
model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)
```

In [8]:
```
model.fit(X_train, y_train)
```

Out[8]:

▼ RandomForestClassifier

▶ Parameters

In [9]:
```
y_pred = model.predict(X_test)
```

In [10]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Accuracy: 0.6776923076923077

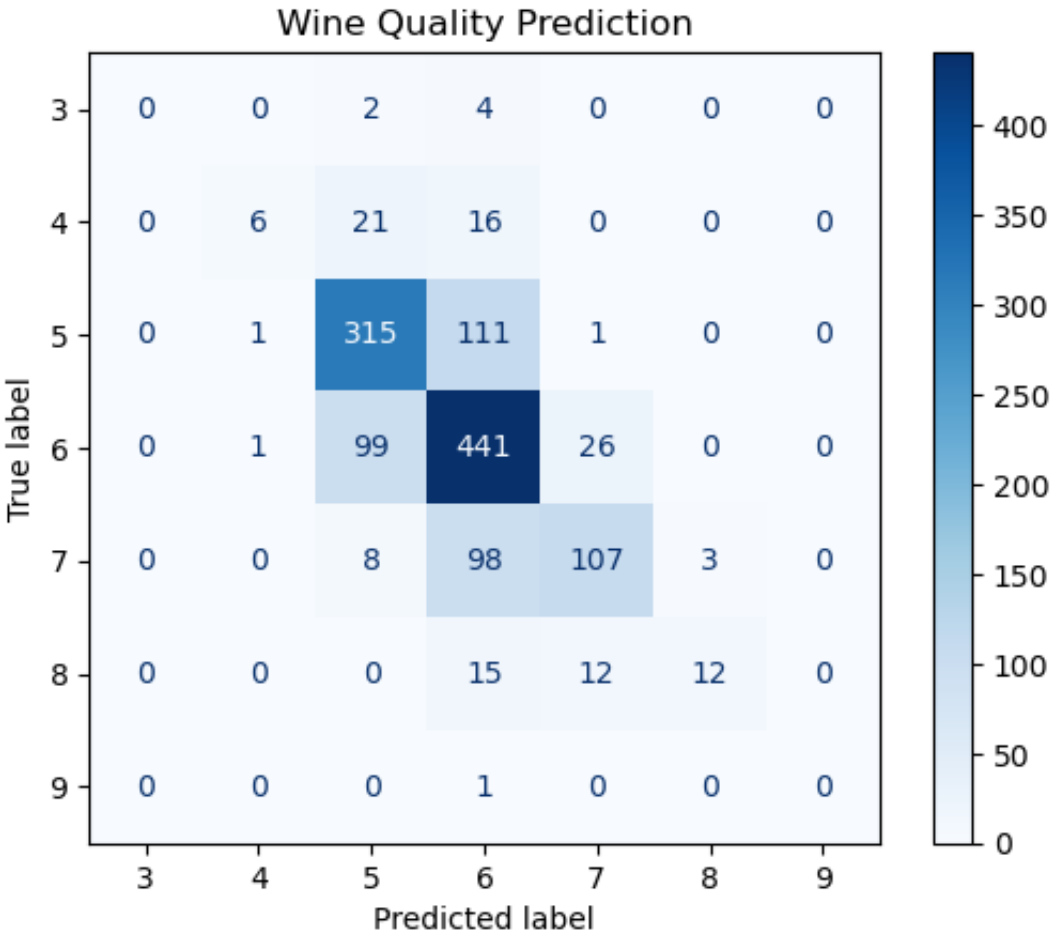
Classification Report:

              precision    recall  f1-score   support

           3       0.00      0.00      0.00         6
           4       0.75      0.14      0.24        43
           5       0.71      0.74      0.72       428
           6       0.64      0.78      0.70       567
           7       0.73      0.50      0.59       216
           8       0.80      0.31      0.44        39
           9       0.00      0.00      0.00         1

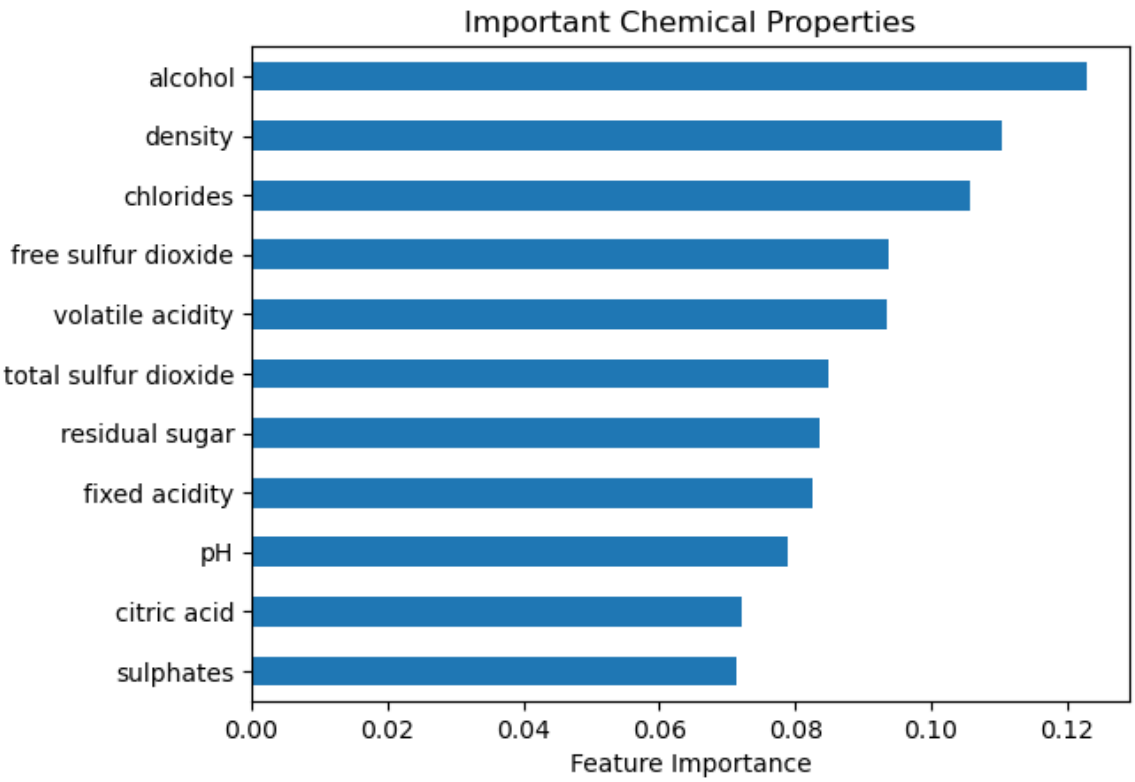
    accuracy                           0.68      1300
   macro avg       0.52      0.35      0.39      1300
weighted avg       0.68      0.68      0.66      1300
```

In [12]:
```
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred, cmap="Blues"
)
plt.title("Wine Quality Prediction")
plt.show()
```



In [13]:
```
importance = pd.Series(
    model.feature_importances_,
    index=X.columns
).sort_values(ascending=True)

importance.plot(kind="barh")
plt.title("Important Chemical Properties")
plt.xlabel("Feature Importance")
plt.show()
```



4. Develop a Support Vector Machine (SVM) classification model to predict whether a tumor is malignant or benign based on various features derived from digitized images of fine needle aspirate (FNA) of breast mass.

In [14]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)
```

In [15]:
```
df = pd.read_csv("Breast_cancer.csv")
```

In [16]:
```
df.sample(10)
```

Out[16]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	are
534	919537	B	10.96	17.62	70.79	
545	922576	B	13.62	23.23	87.19	
62	858986	M	14.25	22.15	96.42	
182	873701	M	15.70	20.31	101.20	
108	86355	M	22.27	19.67	152.80	
551	923780	B	11.13	22.44	71.49	
209	8810436	B	15.27	12.91	98.17	
219	88119002	M	19.53	32.47	128.00	
276	8911230	B	11.33	14.16	71.79	
476	911654	B	14.20	20.53	92.41	

10 rows × 33 columns

In [17]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   id                       569 non-null    int64  
 1   diagnosis                569 non-null    object 
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

In [18]:
```
df.shape
```

Out[18]:
```
(569, 33)
```

In [19]:
```
df.isnull().sum()
```

Out[19]:
```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
Unnamed: 32              569
dtype: int64
```

In [20]:
```
df = df.drop(columns=["id", "Unnamed: 32"], errors="ignore")
```

In [21]:
```
df.head()
```

Out[21]:

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smo
0	M	17.99	10.38	122.80	1001.0	
1	M	20.57	17.77	132.90	1326.0	
2	M	19.69	21.25	130.00	1203.0	
3	M	11.42	20.38	77.58	386.1	
4	M	20.29	14.34	135.10	1297.0	

5 rows × 31 columns

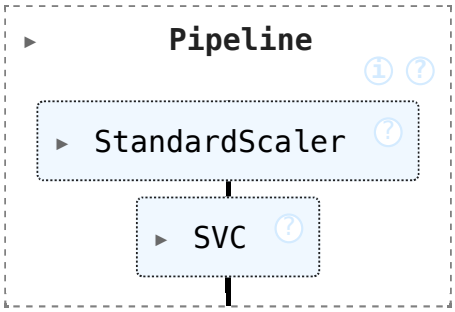
In [22]:
```
X = df.drop(columns=["diagnosis"])
y = df["diagnosis"]
```

In [23]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

In [24]:
```
model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1.0, gamma="scale")
)
```

In [25]:
```
model.fit(X_train, y_train)
```

Out[25]:



In [26]:
```
y_pred = model.predict(X_test)
```

In [27]:
```
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2),
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 97.37 %

Classification Report:
              precision    recall  f1-score   support

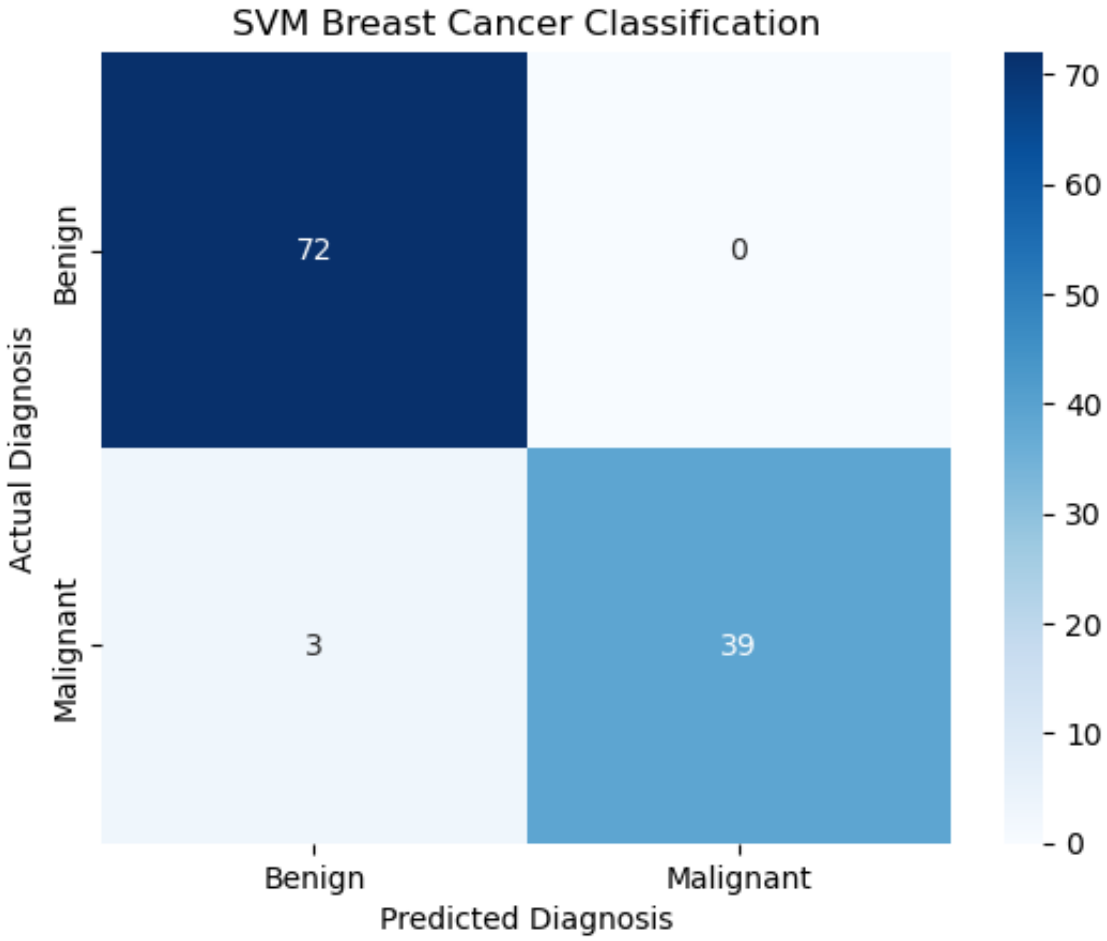
           B       0.96      1.00      0.98        72
           M       1.00      0.93      0.96        42

    accuracy                           0.97       114
   macro avg       0.98      0.96      0.97       114
weighted avg       0.97      0.97      0.97       114
```

In [28]:
```
cm = confusion_matrix(y_test, y_pred, labels=["B", "M"])

sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.xlabel("Predicted Diagnosis")
plt.ylabel("Actual Diagnosis")
plt.title("SVM Breast Cancer Classification")
plt.show()
```



In [29]:
```
sample = X_test.iloc[[0]]
print("Actual diagnosis:", y_test.iloc[0])
print("Predicted diagnosis:", model.predict(sample)[0])
```

```
Actual diagnosis: B
Predicted diagnosis: B
```

5. Develop a Random Forest classification model to predict the forest cover type based on cartographic variables. Handle missing data and perform feature scaling. Evaluate the model using accuracy, precision, recall, and F1 score.

In [30]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score,
    classification_report, confusion_matrix
)
```

In [31]:
```
df = pd.read_csv("covtype.csv")
```

In [32]:
```
df.sample(5)
```

Out[32]:

	Elevation	Aspect	Slope	Horizontal_Distance_To_Hydrology	Vertic
197613	3115	63	7	124	
570193	2619	81	17	255	
247918	2967	222	11	210	
451433	3194	135	3	30	
284759	2451	225	0	190	

5 rows × 55 columns

In [33]:
```
df.isnull().sum()
```

Out[33]:
```
Elevation                           0
Aspect                              0
Slope                               0
Horizontal_Distance_To_Hydrology    0
Vertical_Distance_To_Hydrology      0
Horizontal_Distance_To_Roadways     0
Hillshade_9am                       0
Hillshade_Noon                      0
Hillshade_3pm                       0
Horizontal_Distance_To_Fire_Points  0
Wilderness_Area1                    0
Wilderness_Area2                    0
Wilderness_Area3                    0
Wilderness_Area4                    0
Soil_Type1                          0
Soil_Type2                          0
Soil_Type3                          0
Soil_Type4                          0
Soil_Type5                          0
Soil_Type6                          0
Soil_Type7                          0
Soil_Type8                          0
Soil_Type9                          0
Soil_Type10                         0
Soil_Type11                         0
Soil_Type12                         0
Soil_Type13                         0
Soil_Type14                         0
Soil_Type15                         0
Soil_Type16                         0
Soil_Type17                         0
Soil_Type18                         0
Soil_Type19                         0
Soil_Type20                         0
Soil_Type21                         0
Soil_Type22                         0
```

```
Soil_Type23                         0
Soil_Type24                         0
Soil_Type25                         0
Soil_Type26                         0
Soil_Type27                         0
Soil_Type28                         0
Soil_Type29                         0
Soil_Type30                         0
Soil_Type31                         0
Soil_Type32                         0
Soil_Type33                         0
Soil_Type34                         0
Soil_Type35                         0
Soil_Type36                         0
Soil_Type37                         0
Soil_Type38                         0
Soil_Type39                         0
Soil_Type40                         0
Cover_Type                          0
dtype: int64
```

In [34]:
```
df.shape
```

Out[34]:
```
(581012, 55)
```

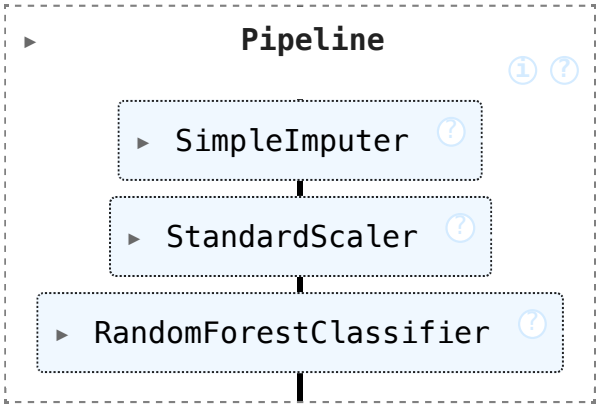
In [35]:
```
X = df.drop(columns=["Cover_Type"])
y = df["Cover_Type"]
```

In [36]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

In [37]:
```
model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    ))
])
```

In [38]:
```
model.fit(X_train, y_train)
```

Out[38]:



In [39]:
```
y_pred = model.predict(X_test)
```

In [40]:
```
print("\nModel Evaluation")
print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(
    y_test, y_pred, average="weighted", zero_division=0
), 4))
print("Recall   :", round(recall_score(
    y_test, y_pred, average="weighted", zero_division=0
), 4))
print("F1 Score :", round(f1_score(
    y_test, y_pred, average="weighted", zero_division=0
), 4))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Model Evaluation
Accuracy : 0.9542
Precision: 0.9543
Recall   : 0.9542
F1 Score : 0.9539

Classification Report:
              precision    recall  f1-score   support

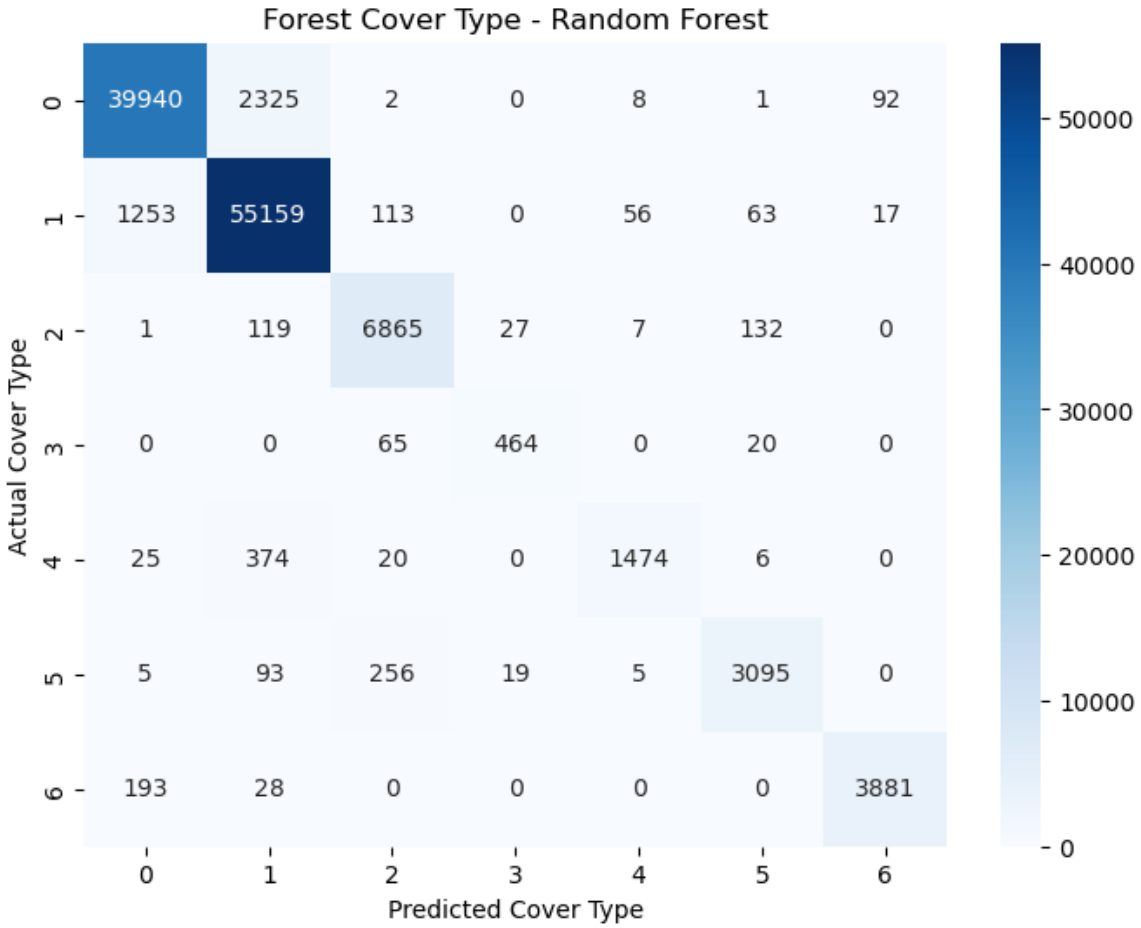
           1       0.96      0.94      0.95     42368
           2       0.95      0.97      0.96     56661
           3       0.94      0.96      0.95      7151
           4       0.91      0.85      0.88       549
           5       0.95      0.78      0.85      1899
           6       0.93      0.89      0.91      3473
           7       0.97      0.95      0.96      4102

    accuracy                           0.95    116203
   macro avg       0.95      0.90      0.92    116203
weighted avg       0.95      0.95      0.95    116203
```

In [41]:
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted Cover Type")
```

```
plt.ylabel("Actual Cover Type")
plt.title("Forest Cover Type – Random Forest")
plt.show()
```



6. Utilize the K-Means clustering algorithm to segment customers based on their purchasing behaviour. Apply the Elbow Method to determine the optimal number of clusters that best represent distinct customer segments for targeted marketing strategies.

In [42]:
```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.cluster import KMeans
```

In [44]:
```
df = pd.read_csv("Customer_segmentation.csv")
print("Dataset shape:", df.shape)
```

```
Dataset shape: (8068, 11)
```

In [45]:
```
df.head()
```

Out[45]:

	ID	Gender	Ever_Married	Age	Graduated	Profession	Work_Exp
0	462809	Male	No	22	No	Healthcare	
1	462643	Female	Yes	38	Yes	Engineer	
2	466315	Female	Yes	67	Yes	Engineer	
3	461735	Male	Yes	67	Yes	Lawyer	
4	462669	Female	Yes	40	Yes	Entertainment	

In [46]:
```
X = df.drop(columns=["ID", "Segmentation"], errors="ignore")
```

In [47]:
```
num_cols = X.select_dtypes(include=["number"]).columns.tolist()
cat_cols = X.select_dtypes(exclude=["number"]).columns.tolist()
```

In [48]:
```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

In [49]:
```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

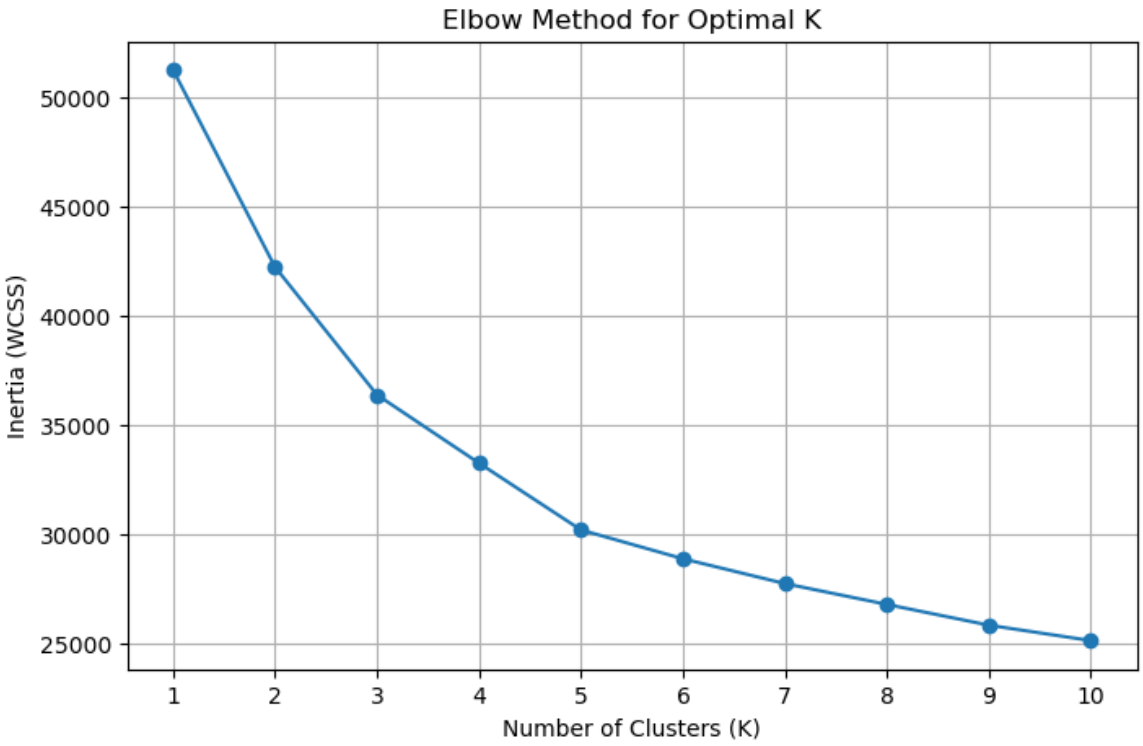
In [50]:
```
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, num_cols),
    ("cat", categorical_pipeline, cat_cols)
])
```

In [51]:
```
X_scaled = preprocessor.fit_transform(X)
```

In [52]:
```
inertia = []
k_values = range(1, 11)

for k in k_values:
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    model.fit(X_scaled)
    inertia.append(model.inertia_)
```

In [53]:
```
plt.figure(figsize=(8, 5))
plt.plot(k_values, inertia, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia (WCSS)")
plt.title("Elbow Method for Optimal K")
plt.xticks(list(k_values))
plt.grid(True)
plt.show()
```



In [57]:
```
optimal_k = 5 # Change this based on your graph

kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(X_scaled)
```

In [58]:
```
df["Cluster"] = clusters

print("\nCustomer count in each cluster:")
print(df["Cluster"].value_counts().sort_index())
```

```
Customer count in each cluster:
Cluster
0    1572
1    2195
2    1436
3    1623
4    1242
Name: count, dtype: int64
```

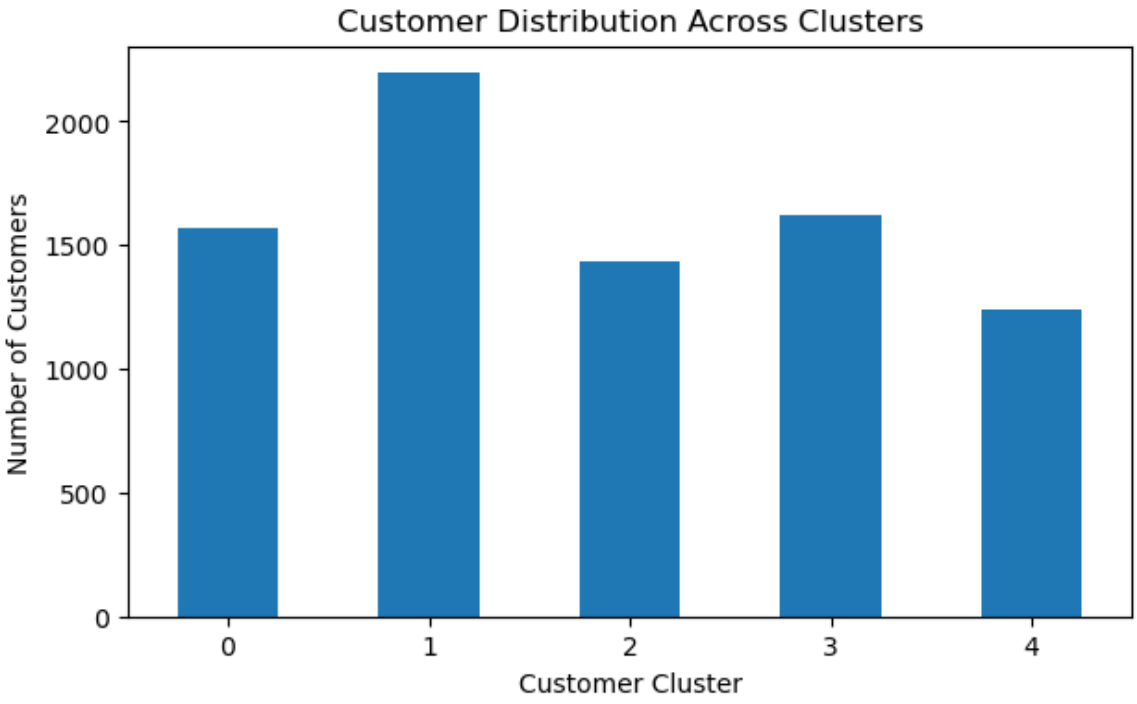
In [59]:
```
print("\nAverage numerical values by cluster:")
print(df.groupby("Cluster")[num_cols].mean().round(2))

print("\nSpending score distribution by cluster:")
print(pd.crosstab(df["Cluster"], df["Spending_Score"]))
```

```
Average numerical values by cluster:
           Age  Work_Experience  Family_Size
Cluster
0        26.07             1.46         4.46
1        46.76             1.15         3.40
2        37.44             8.59         2.43
3        39.33             1.06         1.66
4        72.05             0.98         1.88

Spending score distribution by cluster:
Spending_Score  Average  High   Low
Cluster
0                    14    26  1532
1                  1536   446   213
2                   319   149   968
3                     0    15  1608
4                   105   580   557
```

In [60]:
```
df["Cluster"].value_counts().sort_index().plot(
    kind="bar", figsize=(7, 4)
)
plt.xlabel("Customer Cluster")
plt.ylabel("Number of Customers")
plt.title("Customer Distribution Across Clusters")
plt.xticks(rotation=0)
plt.show()
```



7. Use K-Medoids clustering to segment wholesale customers based on their annual spending on different product categories. Identify the optimal number of clusters to effectively group customers with similar purchasing behaviours.

In [64]:
```
pip install scikit-learn-extra
```

```
Collecting scikit-learn-extra
  Downloading scikit-learn-extra-0.3.0.tar.gz (818 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 819.0/819.0 kB 815.4 k
B/s  0:00:011m901.2 kB/s eta 0:00:01
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: numpy>=1.13.3 in ./jupyter_notebook/a
naconda3/lib/python3.13/site-packages (from scikit-learn-extra) (2.
3.5)
Requirement already satisfied: scipy>=0.19.1 in ./jupyter_notebook/a
naconda3/lib/python3.13/site-packages (from scikit-learn-extra) (1.1
6.3)
Requirement already satisfied: scikit-learn>=0.23.0 in ./jupyter_not
ebook/anaconda3/lib/python3.13/site-packages (from scikit-learn-extr
a) (1.7.2)
Requirement already satisfied: joblib>=1.2.0 in ./jupyter_notebook/a
naconda3/lib/python3.13/site-packages (from scikit-learn>=0.23.0->sc
ikit-learn-extra) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in ./jupyter_not
ebook/anaconda3/lib/python3.13/site-packages (from scikit-learn>=0.2
3.0->scikit-learn-extra) (3.5.0)
Building wheels for collected packages: scikit-learn-extra
  Building wheel for scikit-learn-extra (pyproject.toml) ... done
  Created wheel for scikit-learn-extra: filename=scikit_learn_extra-
0.3.0-cp313-cp313-macosx_12_0_arm64.whl size=383911 sha256=2aadec54a
03b8f354e92bc11823fdd41a138aef313a818322e17999c8255219b
  Stored in directory: /Users/sudambhagwanwankhede/Library/Caches/pi
p/wheels/17/4b/b8/6b6711681d0981b110c9cc91ad6d1ebd88adf1547e1da301fc
Successfully built scikit-learn-extra
Installing collected packages: scikit-learn-extra
Successfully installed scikit-learn-extra-0.3.0
Note: you may need to restart the kernel to use updated packages.
```

In [65]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn_extra.cluster import KMedoids
```

In [67]:
```
df = pd.read_csv("Wholesale customers data.csv")
```

In [68]:
```
df.shape
```

Out[68]:
```
(440, 8)
```

In [70]:
```
df.sample(5)
```

Out[70]:

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	I
183	1	3	36847	43950	20170	36534	239	
320	1	2	1774	3696	2280	514	275	
176	1	3	45640	6958	6536	7368	1532	
299	1	2	444	879	2060	264	290	
258	1	1	56083	4563	2124	6422	730	

In [71]:
```
features = [
    "Fresh", "Milk", "Grocery",
    "Frozen", "Detergents_Paper", "Delicassen"
]
X = df[features]
```

In [72]:
```
imputer = SimpleImputer(strategy="median")
X_imputed = imputer.fit_transform(X)
```

In [73]:
```
X_log = np.log1p(X_imputed)

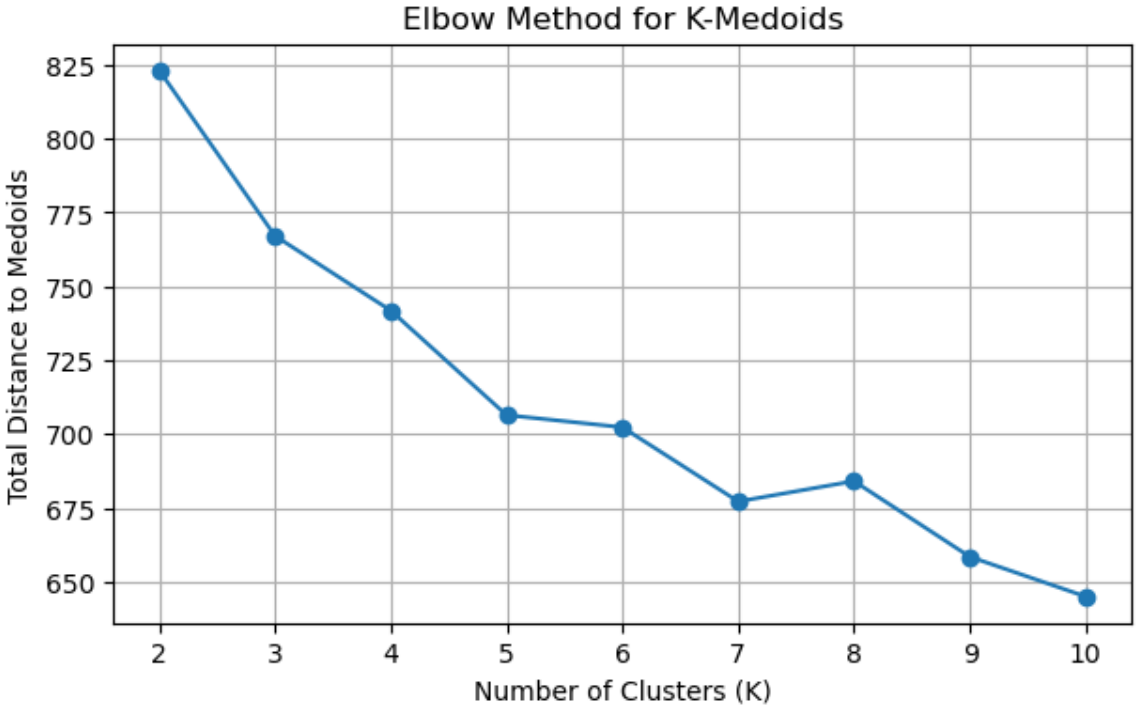
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_log)
```

In [74]:
```
costs = []
sil_scores = []
k_values = range(2, 11)

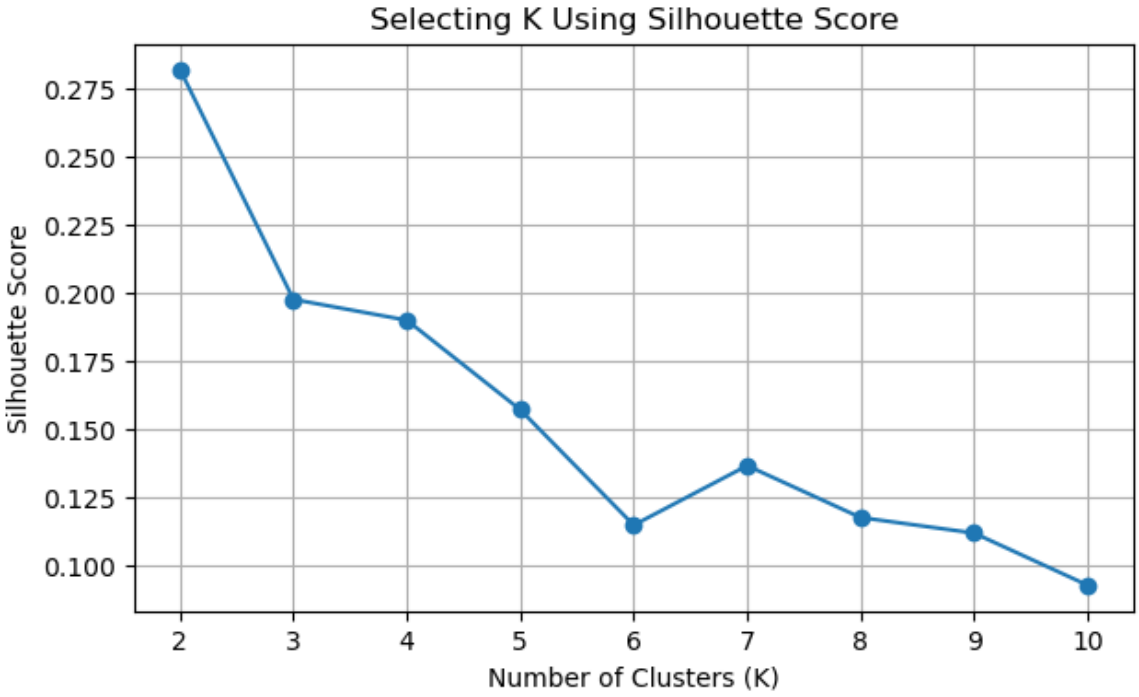
for k in k_values:
    model = KMedoids(
        n_clusters=k,
        random_state=42,
        method="alternate"
    )
    labels = model.fit_predict(X_scaled)

    costs.append(model.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))
```

In [75]:
```
plt.figure(figsize=(7, 4))
plt.plot(list(k_values), costs, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Total Distance to Medoids")
plt.title("Elbow Method for K-Medoids")
plt.xticks(list(k_values))
plt.grid(True)
plt.show()
```



In [76]:
```
plt.figure(figsize=(7, 4))
plt.plot(list(k_values), sil_scores, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("Selecting K Using Silhouette Score")
plt.xticks(list(k_values))
plt.grid(True)
plt.show()
```



In [77]:
```
optimal_k = list(k_values)[np.argmax(sil_scores)]
print("Optimal number of clusters:", optimal_k)
print("Best silhouette score:", round(max(sil_scores), 4))
```

```
Optimal number of clusters: 2
Best silhouette score: 0.2822
```

In [78]:
```
final_model = KMedoids(
    n_clusters=optimal_k,
    random_state=42,
    method="alternate"
)

df["Cluster"] = final_model.fit_predict(X_scaled)
```

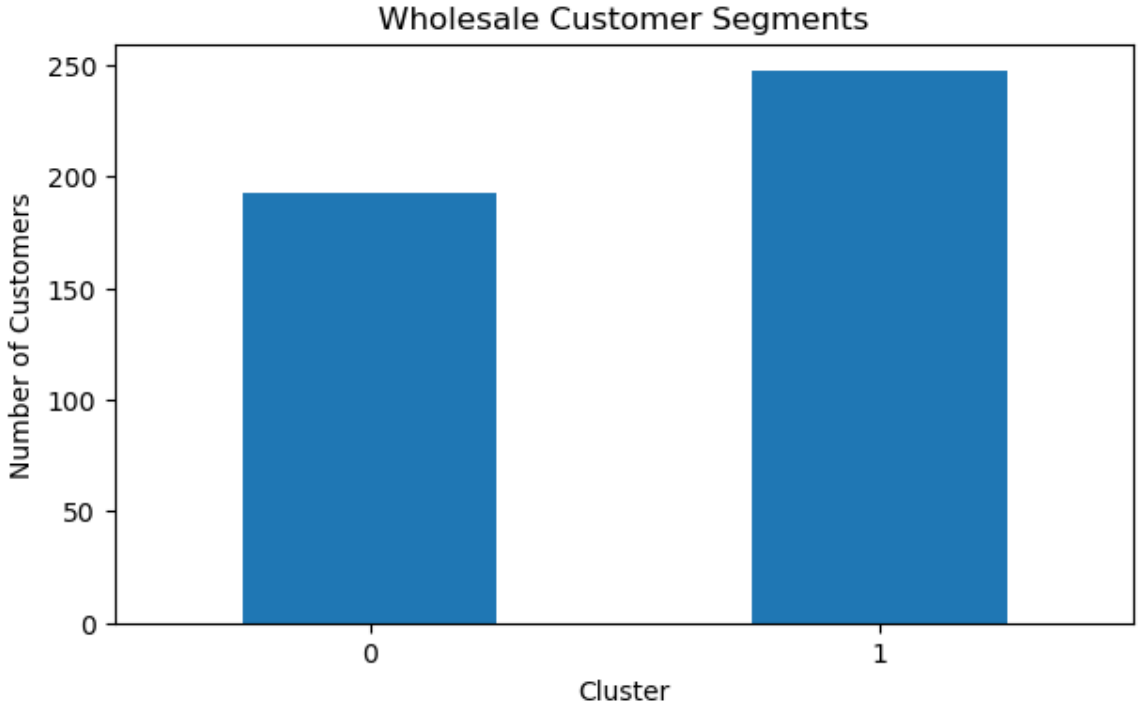
In [79]:
```
print("\nCustomers in each cluster:")
print(df["Cluster"].value_counts().sort_index())

print("\nAverage annual spending by cluster:")
print(df.groupby("Cluster")[features].mean().round(2))
```

```
Customers in each cluster:
Cluster
0    193
1    247
Name: count, dtype: int64

Average annual spending by cluster:
            Fresh      Milk   Grocery   Frozen  Detergents_Paper  De
licassen
Cluster
0        10827.85  10226.05  14489.95  2728.88           5887.69
2312.28
1        12916.42   2334.94   2842.11  3339.99            532.52
909.61
```

In [80]:
```
df["Cluster"].value_counts().sort_index().plot(
    kind="bar", figsize=(7, 4)
)
plt.xlabel("Cluster")
plt.ylabel("Number of Customers")
plt.title("Wholesale Customer Segments")
plt.xticks(rotation=0)
plt.show()
```


Wholesale Customer Segments
Number of Customers
250
200
150
100
50
0
0
1
Cluster

In [ ]: